

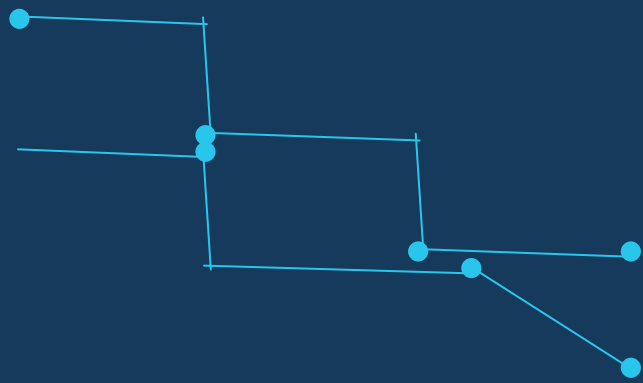
EQUIPE ROSA

CORE PROJECT

Inteligência Artificial

- Felipe Gabriel Sasaki
- Leonardo Alves
- João Pedro Abreu





NOSSA ESCOLHA:

**PEDRA, PAPEL E
TESOURA
(JOKENPÔ)**

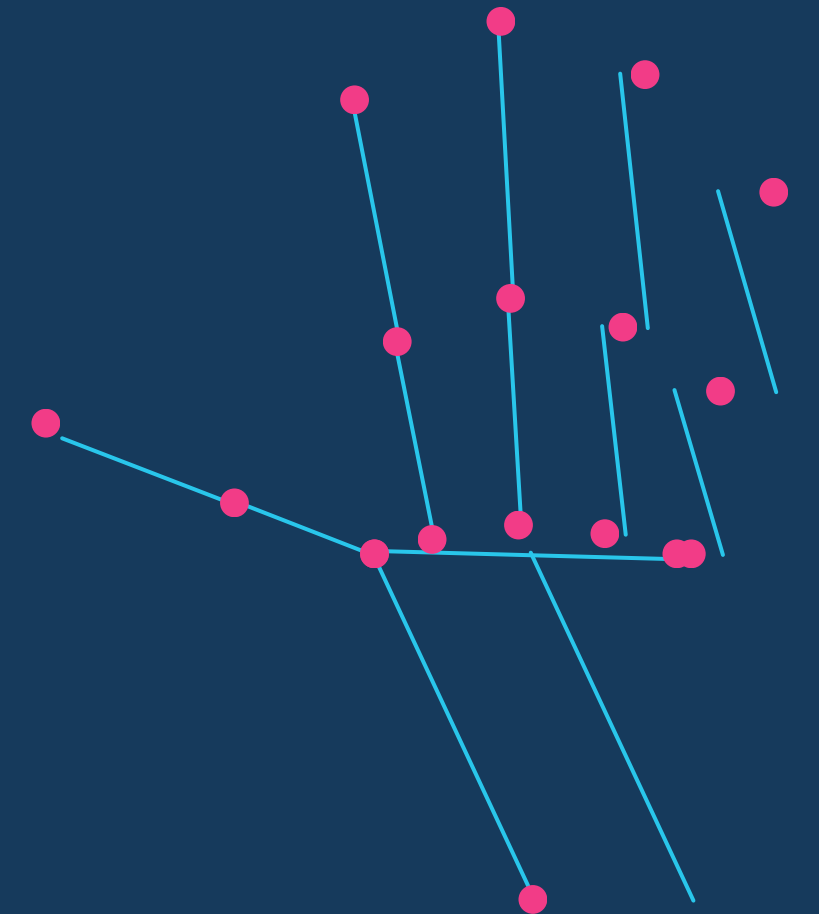
Como permitir que um computador reconheça em tempo real os gestos de mãos humanas e jogue contra um participante — usando apenas uma câmera e visão computacional.





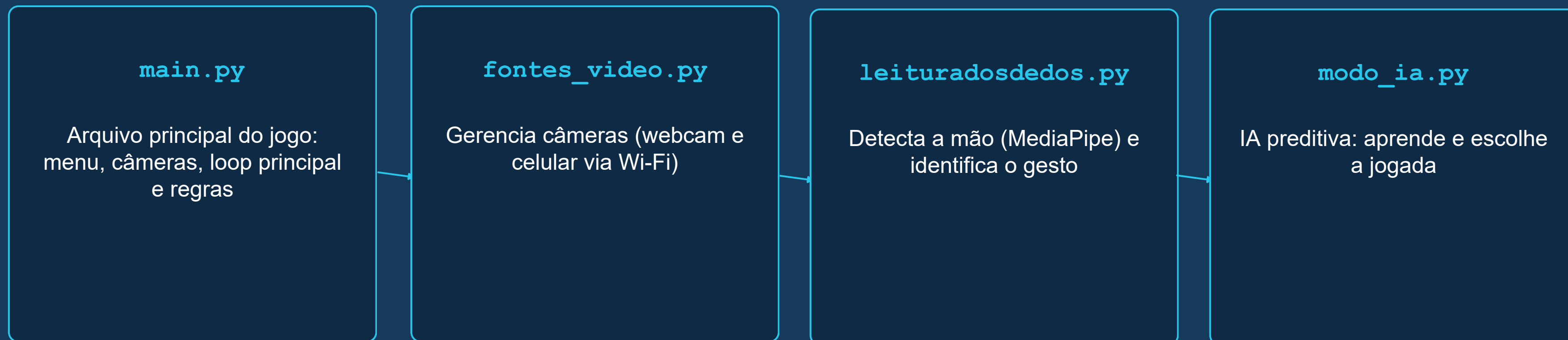
VISÃO COMPUTACIONAL

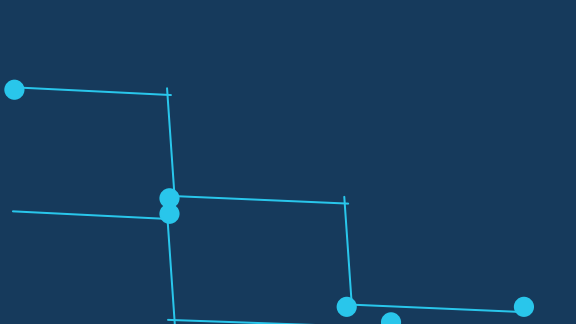
- **Inteligência Artificial**
→ Interpretação, processamento e compreensão do mundo real por um sistema computacional
- **Reconhecimento de gestos**
→ Feito com o Google MediaPipe (Hand Landmarker), modelo pré-treinado que localiza 21 pontos-chave da mão a cada frame



ARQUITETURA DO PROJETO

→ Organização do código em módulos Python





ESQUELETO DA MÃO COLORIDO

Cor por gesto: a função pinta o esqueleto de azul para Pedra, amarelo para Papel e vermelho para Tesoura e com ciano quando não reconhece nada;

Coordenadas ajustadas: ela converte os pontos normalizados (0 a 1) para pixels e soma o deslocamento da ROI, então o desenho fica no lugar certo tanto no frame inteiro quanto no recorte;

Desenho da mão: linhas ligam as articulações e cada ponto é um círculo branco com centro colorido, o que dá o feedback visual em tempo real para o jogador.



GitHub

```
def desenhar_esqueleto_hud(frame, landmarks, offset_x, offset_y, roi_w, roi_h, gesto):

    gesto_lower = gesto.lower()

    if "pedra" in gesto_lower:
        cor_linha = (255, 128, 0)
    elif "papel" in gesto_lower:
        cor_linha = (0, 255, 255)
    elif "tesoura" in gesto_lower:
        cor_linha = (0, 0, 255)
    else:
        cor_linha = (255, 255, 0)

    pontos_px = [
        (int(p.x * roi_w) + offset_x, int(p.y * roi_h) + offset_y)
        for p in landmarks
    ]
    for a, b in CONEXOES_MAO:
        cv2.line(frame, pontos_px[a], pontos_px[b], cor_linha, 2)

    for x, y in pontos_px:
        cv2.circle(frame, (x, y), 5, (255, 255, 255), -1)
        cv2.circle(frame, (x, y), 3, cor_linha, -1)
```



CÂMERA EM SEGUNDO PLANO

- **Leitura contínua:** uma thread separada lê os frames o tempo todo, então o jogo nunca espera pela câmera e não acumula atraso no buffer;
- **Frame sempre atual:** a thread guarda só o último frame, protegido por um lock, e o jogo recebe uma cópia dele para não corromper a imagem em uso;
- **Serve para tudo:** a mesma classe funciona com a webcam e com o celular via Wi-Fi, e o método `release()` encerra a thread e libera a câmera.



GitHub

```
class CameraAsync:
    """Lê os frames continuamente em uma thread separada para evitar acúmulo no buffer."""
    def __init__(self, src):
        self.cap = cv2.VideoCapture(src)

        self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)

        self.ret, self.frame = self.cap.read()
        self.rodando = True
        self.lock = threading.Lock()

        if self.cap.isOpened():
            threading.Thread(target=self._atualizar, daemon=True).start()

    def _atualizar(self):
        while self.rodando:
            ret, frame = self.cap.read()
            with self.lock:
                self.ret = ret
                self.frame = frame

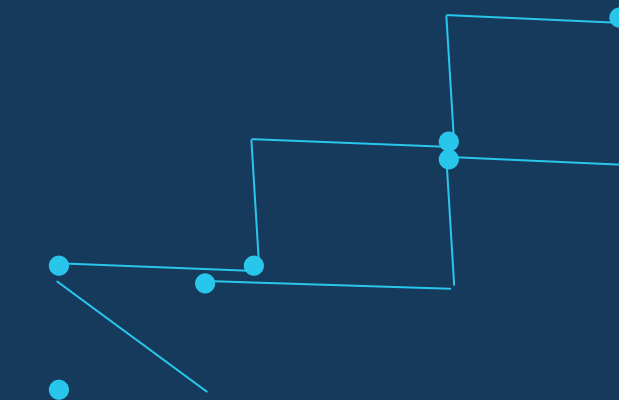
    def read(self):
        with self.lock:
            if self.frame is not None:
                return self.ret, self.frame.copy()
            return self.ret, None

    def release(self):
        self.rodando = False
        self.cap.release()

    def isOpened(self):
        return self.cap.isOpened()
```

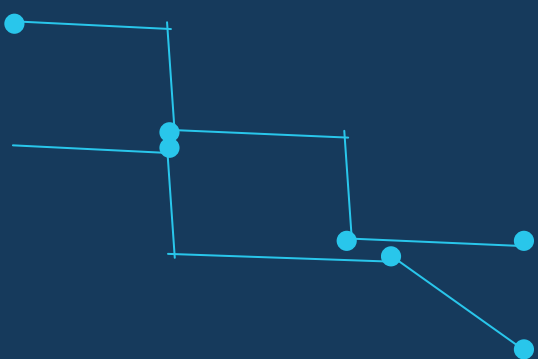


DETECÇÃO DA MÃO



→ HandLandmarker (Google MediaPipe)

- 21 pontos (landmarks) identificados em cada mão, por frame
- `RunningMode.VIDEO` — processamento contínuo do vídeo em tempo real
- Confiança mínima de 0.6 para detecção, presença e rastreamento
- `num_hands` configurável: 1 mão (modo IA) ou 2 mãos (modo PvP na mesma câmera)
- Esqueleto desenhado sobre o vídeo com `cv2.line` e `cv2.circle` a cada frame



IDENTIFICAÇÃO DA MÃO

- Configuração do pipeline do Mediapipe para detectar e rastrear as mãos.
- Localização do arquivo de modelo
- Parâmetros do MediaPipe
- Detector e Temporizador



GitHub

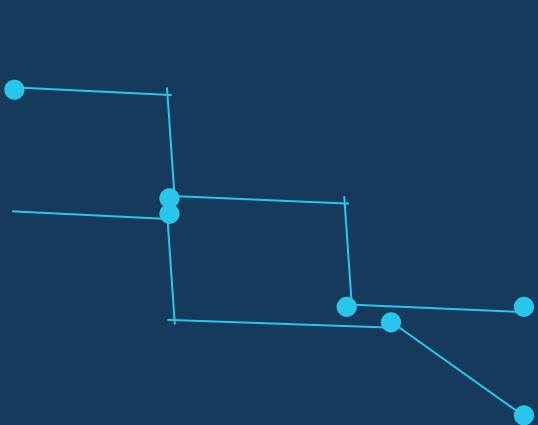


```
class DetectorMao:
    def __init__(self, num_hands=1):
        pasta_atual = os.path.dirname(os.path.abspath(__file__))
        caminho_modelo = os.path.join(pasta_atual, 'hand_landmarker.task')

        if not os.path.exists(caminho_modelo):
            raise FileNotFoundError("Baixe o arquivo hand_landmarker.task para a pasta do projeto!")

        options = vision.HandLandmarkerOptions(
            base_options=BaseOptions(model_asset_path=caminho_modelo),
            running_mode=vision.RunningMode.VIDEO,
            num_hands=num_hands,
            min_hand_detection_confidence=0.6,
            min_hand_presence_confidence=0.6,
            min_tracking_confidence=0.6,
        )
        self._landmarker = vision.HandLandmarker.create_from_options(options)
        self._relogio = itertools.count(start=0, step=33)
```

** versão distância-ao-pulso: funciona com a mão em qualquer rotação*



O CÓDIGO

Pega os pontos da mão identificados pela câmera e descobre se o jogador fez Pedra, Papel ou Tesoura.

- Pulso e Id dos dedos
- Verificação de dedos abertos
- Lógica de classificação das jogadas



GitHub

```
# leituradosdedos.py
```

```
def identificar_gesto(hand_landmarks):  
    pulso = hand_landmarks[0]  
    tip_ids = [8, 12, 16, 20]  
    pip_ids = [6, 10, 14, 18]  
    dedos_abertos = []
```

```
    for tip, pip in zip(tip_ids, pip_ids):  
        d_ponta = distancia(hand_landmarks[tip], pulso)  
        d_meio = distancia(hand_landmarks[pip], pulso)  
        dedos_abertos.append(1 if d_ponta > d_meio else 0)
```

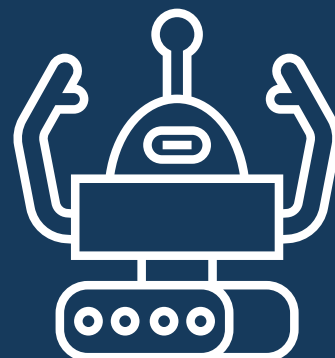
```
    total = sum(dedos_abertos)  
    if total == 0: return "Pedra"  
    if total == 4: return "Papel"  
    if dedos_abertos[0:2] == [1,1] and sum(dedos_abertos[2:]) == 0:  
        return "Tesoura"  
    return "Desconhecido"
```

** versão distância-ao-pulso: funciona com a mão em qualquer rotação*

MEMORIZAÇÃO DA IA



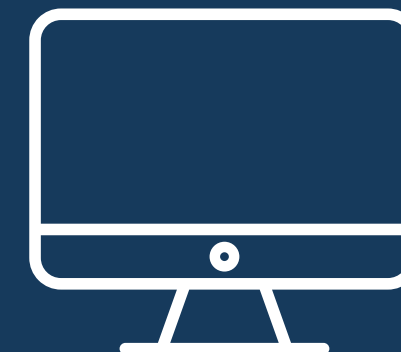
RECONHECIMENTO
DA JOGADA



JOGADOR
VS
MÁQUINA

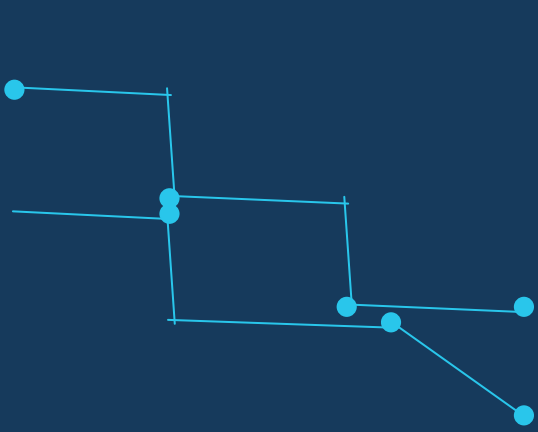


JOGADOR
VS
JOGADOR



INTERFACE
GRÁFICA





IA Preditiva

Decisões da IA com base em jogadas anteriores

Memorização dos dados



GitHub

```
def decidir_jogada(self):
    """Toma uma decisão estocástica baseada nas probabilidades da Cadeia de Markov"""
    opcoes = ["Pedra", "Papel", "Tesoura"]
    contra_ataque = {"Pedra": "Papel", "Papel": "Tesoura", "Tesoura": "Pedra"}

    if len(self.historico_recente) == 2:
        estado = f"{self.historico_recente[0]},{self.historico_recente[1]}"

        if estado in self.memoria:
            frequencias = self.memoria[estado]
            pesos = [frequencias["Pedra"], frequencias["Papel"], frequencias["Tesoura"]]

            # Sorteia a previsão do humano ponderada pelos pesos estatísticos do JSON
            if sum(pesos) > 0:
                previsao_humano = random.choices(opcoes, weights=pesos, k=1)[0]
                return contra_ataque[previsao_humano]

    # Fallback: Se não houver dados suficientes para o estado atual, escolhe aleatoriamente
    return random.choice(opcoes)
```

** versão distância-ao-pulso: funciona com a mão em qualquer rotação*



- 

* versão distância-ao-pulso: funciona com a mão em qualquer rotação

```

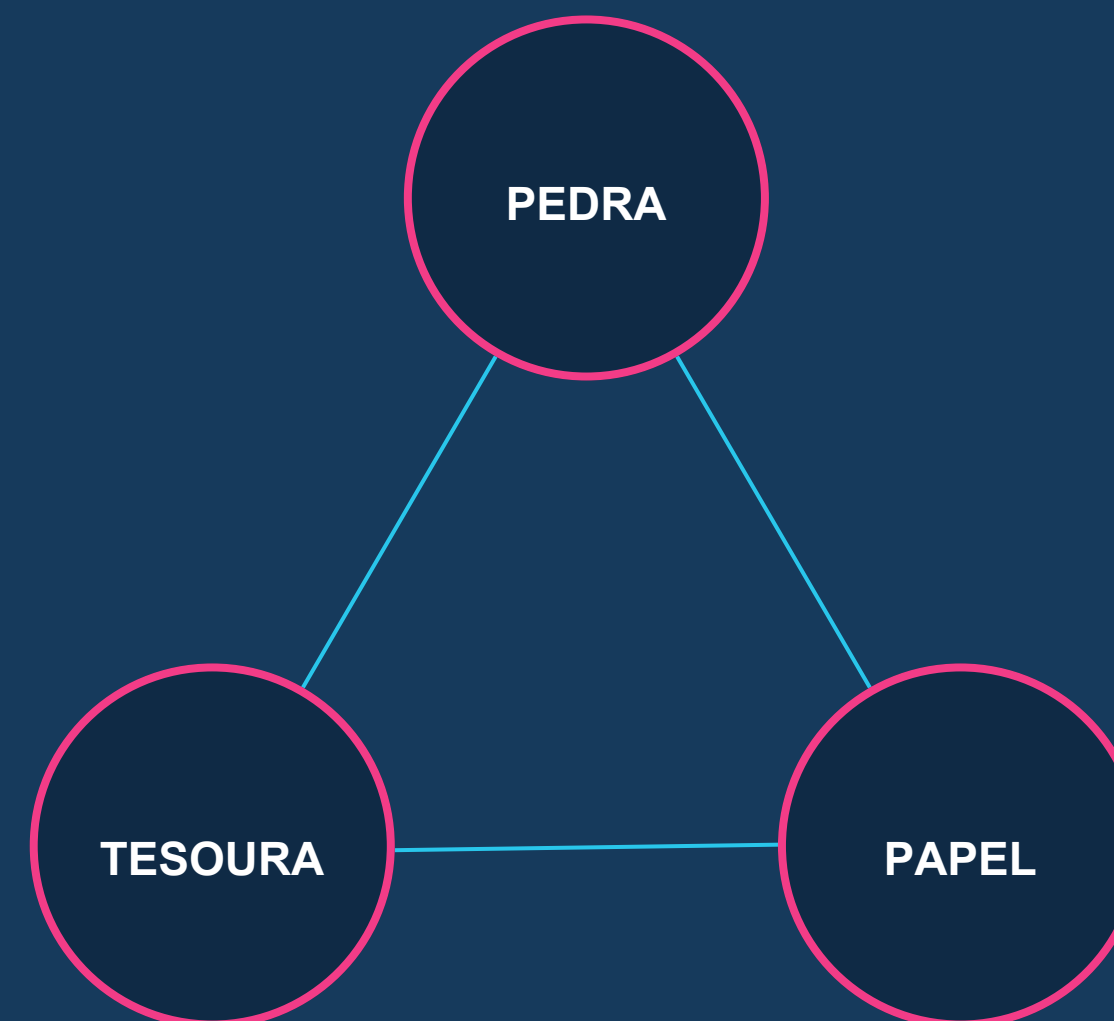
    "Tesoura,Papel": {
        "Pedra": 1,
        "Papel": 0,
        "Tesoura": 0
    },
    "Pedra,Pedra": {
        "Pedra": 2,
        "Papel": 2,
        "Tesoura": 2
    },
    "Pedra,Papel": {
        "Pedra": 1,
        "Papel": 0,
        "Tesoura": 0
    },
    "Papel,Pedra": {
        "Pedra": 1,
        "Papel": 0,
        "Tesoura": 0
    }
}

```



LÓGICA DO JOGO

- Os gestos dos dois lados (jogador vs jogador, ou jogador vs IA) são comparados a cada rodada
- Regra clássica: Pedra vence Tesoura, Tesoura vence Papel, Papel vence Pedra
- Gesto igual em ambos os lados = EMPATE
- Resultado exibido na tela com `cv2.putText`, sobreposto ao vídeo



MODOS DE JOGO

→ Escolhidos no menu inicial (main.py)

1 · JOGADOR VS IA PREDITIVA

- 1 câmera lê a mão do jogador
- IA analisa o histórico e escolhe sua jogada
- Resultado comparado em tempo real

2 · JOGADOR VS JOGADOR (PvP)

- 2 câmeras (webcam + celular via Wi-Fi ou 2 câmeras de celular via Wifi) → 1 mão cada
- ou 1 câmera lendo as 2 mãos lado a lado
- Telas combinadas com combinar_lado_a_lado()





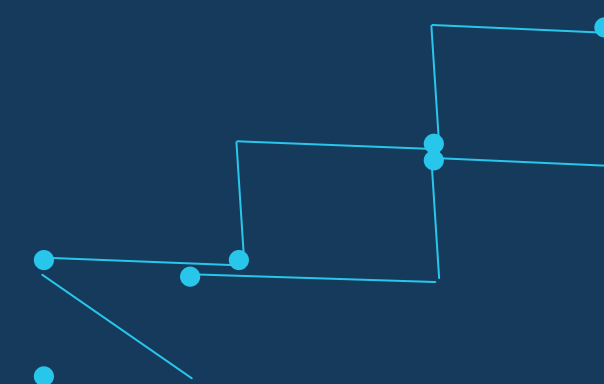
INTERFACE EM TEMPO REAL

- Janela redimensionável via OpenCV (cv2.imshow)
- Esqueleto da mão desenhado sobre o vídeo (pontos + conexões)
- Textos sobrepostos: jogada do jogador, jogada da IA/oponente e resultado
- Tecla ESPAÇO confirma a jogada da rodada
- Tecla 'Q' encerra o jogo e salva a memória da IA



DESAFIOS & APRENDIZADOS

- Reconhecer o gesto independente da rotação da mão (uso de distância euclidiana ao pulso, em vez de comparar só a coordenada Y)
- Sincronizar vídeo de câmeras diferentes (webcam local + celular via Wi-Fi)
- Estudar o funcionamento do MediaPipe a fim de evitar falsos positivos
- Organizar o código em módulos para permitir trabalho colaborativo em equipe (Git/GitHub)
- Implementar uma lógica preditiva para IA em um jogo que só depende de probabilidade
- Criar uma interface gráfica que não prejudicasse a latência





OBRIGADO PELA ATENÇÃO!

Equipe Rosa · Core Project · Inteligência Artificial

